

Advanced Range Data Structures

Sean

August 27, 2026



Outline

- Segment trees
 - Structure
 - Point updates
 - Range queries
- Fenwick trees (BIT)
- Lazy propagation for range updates
- Sparse tables for static queries



Why segment trees?

- We need to answer range queries while values can still change.
- Recomputing every answer is too slow for large inputs.
- A segment tree supports both operations in $O(\log N)$ time.



1 Segment Tree Basics

2 Fenwick Trees

3 Lazy Propagation

4 Sparse Tables



Segment Tree Basics

A dynamic range-maximum problem

Problem (Dynamic Range Maximum Query)

Given an array $a_1, \dots, a_N$, process Q operations of two types:

- 1 l r : report $\max(a_l, a_{l+1}, \dots, a_r)$.
- 2 p x : set $a_p \leftarrow x$.

$$N, Q \leq 5 \times 10^5$$



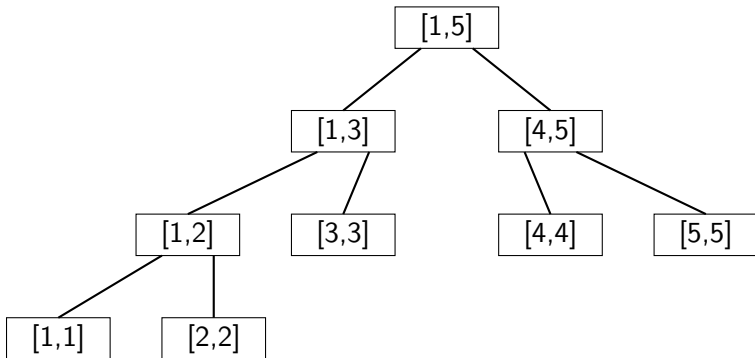
Static preprocessing is not enough

- A sparse table answers static range queries quickly, but an update invalidates many stored intervals.
 - One changed element can affect $\Theta(N)$ precomputed entries.
- We still want to reuse three good ideas:
 - answer a query by combining stored summaries;
 - build a large summary from smaller summaries;
 - split each interval into two roughly equal halves.
- The missing ingredient: each element should belong to only $O(\log N)$ stored summaries.



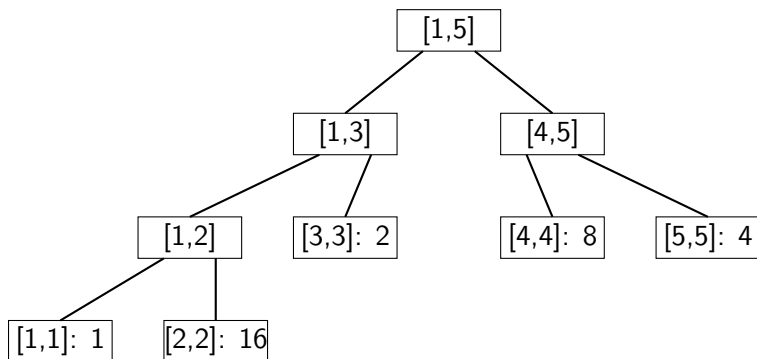
Recursive halves form a balanced hierarchy

- Split every interval near its midpoint until only one-element intervals remain.
- For an array of length 5, the interval hierarchy looks like this:



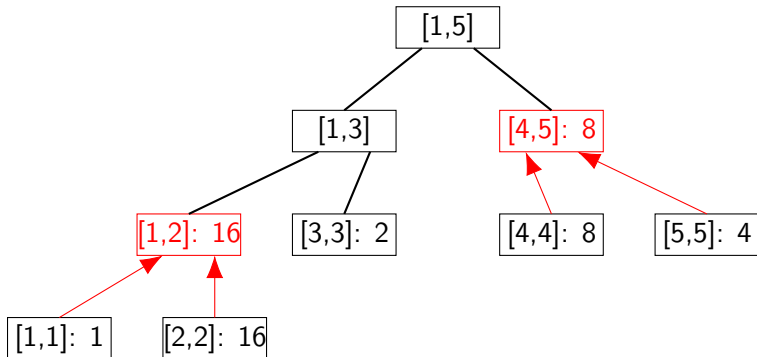
Leaves store the array values

- Each node represents its labeled interval and will store that interval's maximum.
- Start with $a = [1, 16, 2, 8, 4]$ at the leaves.



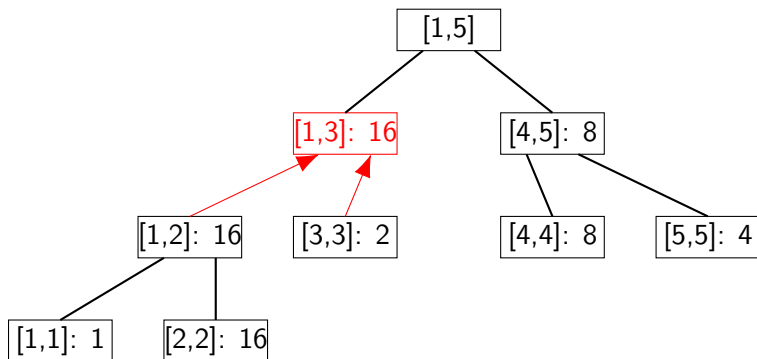
Combine adjacent leaves

- For every internal node, take the maximum of its two children.



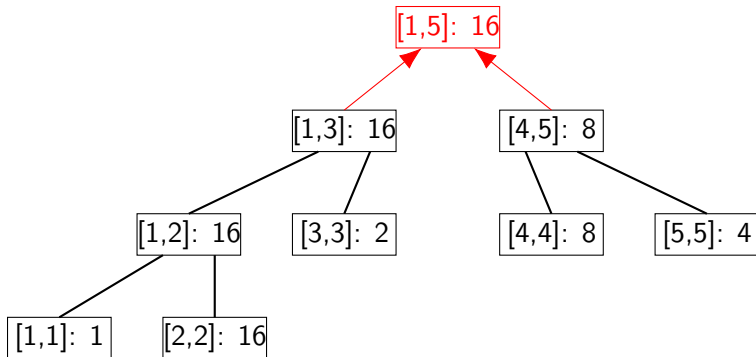
Continue pulling values upward

- Once both children are ready, their parent can be computed in $O(1)$ time.



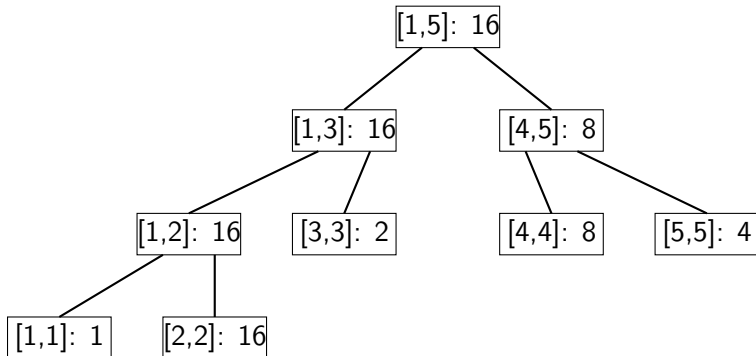
The root summarizes the whole array

- The root stores $\max(a_1, \dots, a_5) = 16$.



Every node maintains the same invariant

- $a = [1, 16, 2, 8, 4]$

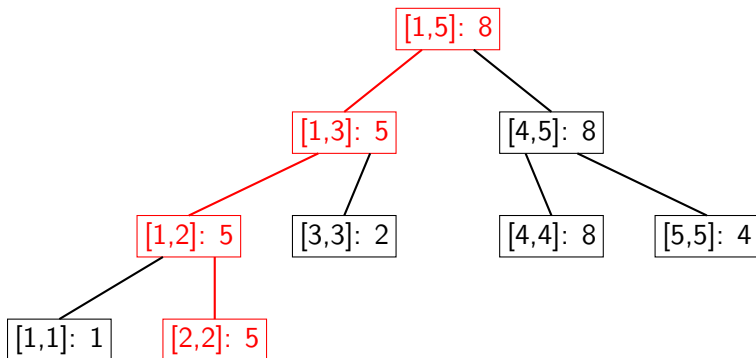


- **Invariant:** a node stores the maximum over exactly its interval.
- A parent's answer is the maximum of its two children's answers.



A point update changes one path

- $a = [1, 16, 2, 8, 4]$



- Only the updated leaf and its ancestors can change.
- That path contains $O(\log N)$ nodes.

The tree stays small and shallow

- There are N leaves and $N - 1$ internal nodes: $2N - 1$ nodes total.
- Each internal node has two children, and the height is $O(\log N)$.
- Building bottom-up takes $O(N)$ time because each node is computed once.
- This balanced hierarchy of interval summaries is a **segment tree**.



Represent a node and its invariant

```
1  const int neg_inf = numeric_limits<int>::lowest();
2  struct Node {
3      Node *lc, *rc;
4      int mx;
5      void pull() { mx = max(lc->mx, rc->mx); }
6  } *root = nullptr;
```

`pull()` restores the invariant after either child changes: the parent maximum becomes the maximum of the two child maxima.



Build recursively in $O(N)$ time

```
1 Node *build(int arr[], int l, int r) { // Build [l, r].
2     Node *res = new Node();
3     if (l == r) { // Leaf
4         res->lc = res->rc = nullptr;
5         res->mx = arr[l];
6     } else {
7         int m = (l + r) / 2;
8         res->lc = build(arr, l, m);           // Left half
9         res->rc = build(arr, m + 1, r);       // Right half
10        res->pull();                          // Restore invariant
11    }
12    return res;
13 }
```



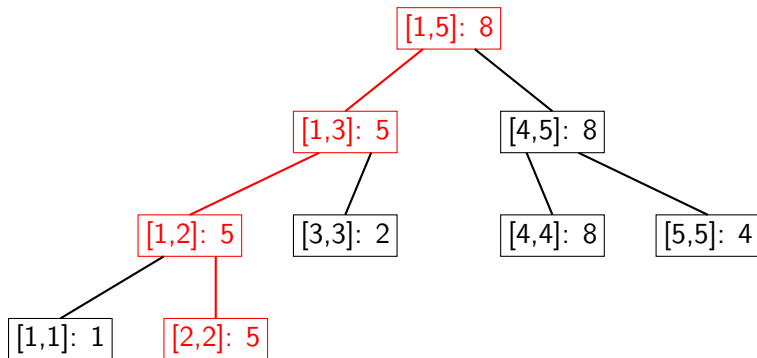
Every recursive call owns one interval

- A call receives both a node and the interval $[l, r]$ represented by that node.
- The base case $l = r$ creates a leaf; otherwise, split at $m = \lfloor (l + r)/2 \rfloor$.
- This deck uses pointers for clarity; an array-based tree follows the same recursion.



Update the leaf, then pull to the root

- $a = [1, 16, 2, 8, 4]$



- Follow the unique path to p and replace its leaf value.
- On the way back, call `pull()` at each red ancestor.
- The update takes $O(\log N)$ time.



A point update has two cases

- Update position p while the current node represents $[L, R]$:
 - 1 If $L = R$, this is the target leaf; assign the new value.
 - 2 Otherwise, recurse into the half containing p , then call `pull()` to repair the current node.
- Exactly one recursive call is made at each level.

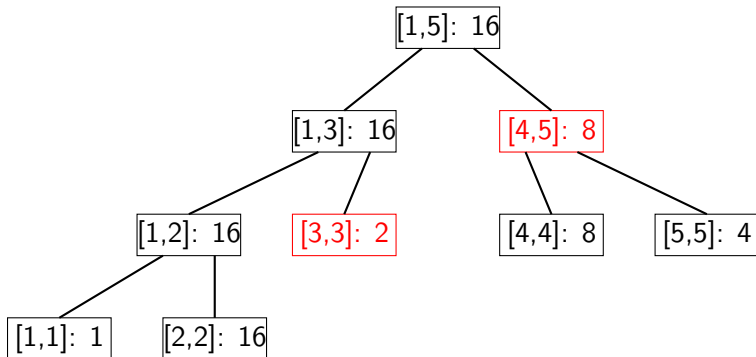
Implement a point update

```
1 void modify(Node *nd, int val, int p, int l, int r) {
2     if (l == r) { // Target leaf
3         nd->mx = val;
4         return;
5     }
6     int m = (l + r) / 2;
7     if (p <= m) // Choose the half containing p.
8         modify(nd->lc, val, p, l, m);
9     else
10        modify(nd->rc, val, p, m + 1, r);
11    nd->pull(); // Repair this node on the way back.
12 }
```



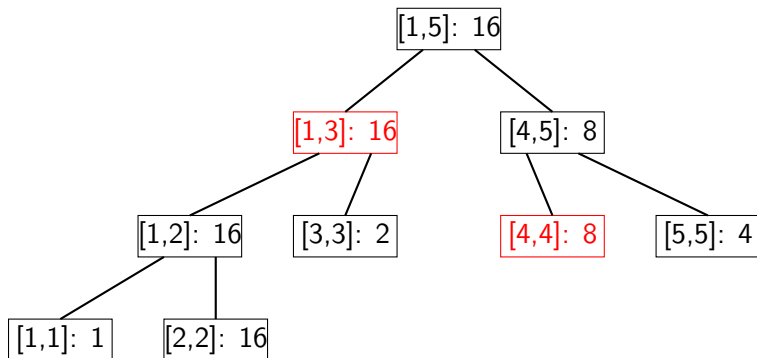
A query combines covered nodes

- For query $[3, 5]$, use the disjoint nodes $[3, 3]$ and $[4, 5]$.
- Their stored maxima combine to $\max(2, 8) = 8$.



Prefer the largest fully covered nodes

- For query $[1, 4]$, use $[1, 3]$ and $[4, 4]$ instead of four leaves.



- The maximum is $\max(16, 8) = 16$ using only two stored summaries.

Stop whenever a node fits completely

- Begin at the root.
- If the current interval lies completely inside the query, use its stored maximum immediately.
- If it is disjoint from the query, ignore it.
- Otherwise, recurse into both children and combine their answers.

A range query has three overlap cases

- Query $[q_l, q_r]$ while the current node represents $[L, R]$:
 - 1 **Fully covered:** return this node's maximum.
 - 2 **Disjoint:** return the identity value $-\infty$.
 - 3 **Partial overlap:** query both children and take the maximum.

Only boundary paths keep branching

- Fully covered subtrees stop immediately; disjoint subtrees also stop immediately.
- At any depth, only the nodes containing the left and right query boundaries can partially overlap.
- Therefore, the query visits $O(\log N)$ relevant nodes in total.
- **A segment tree decomposes any query interval into $O(\log N)$ stored intervals.**



Why the query is $O(\log N)$

- The tree has $O(\log N)$ levels.
- At each level, at most two nodes partially overlap the query: one around q_l and one around q_r .
- Every other visited node is either fully covered or disjoint, so its recursion stops there.
- Thus only $O(1)$ nodes per level continue, for $O(\log N)$ work overall.



Implement a range query

```
1  int query(Node *nd, int ql, int qr, int l, int r) {
2      if (r < ql || l > qr)    // Disjoint
3          return neg_inf;
4      if (ql <= l && r <= qr) // Fully covered
5          return nd->mx;
6      int m = (l + r) / 2;
7      return max(query(nd->lc, ql, qr, l, m),
8                  query(nd->rc, ql, qr, m + 1, r));
9  }
```



1 Segment Tree Basics

2 Fenwick Trees

3 Lazy Propagation

4 Sparse Tables



Fenwick Trees

Dynamic prefix sums need a lighter structure

Problem (Dynamic Range Sum)

Given an array $a_1, \dots, a_N$, process two operations:

- add a value Δ to one position p ;
- report the sum of a range $[l, r]$.

- Prefix sums answer range sums quickly but are expensive to repair after an update.
- A Fenwick tree, also called a Binary Indexed Tree (BIT), supports both operations in $O(\log N)$ time.
- The tree itself uses one array of N stored partial sums; assignments additionally keep the current values.



The lowest set bit determines every stored range

- Use one-based indices and define

$$\text{lowbit}(i) = i \ \& \ -i.$$

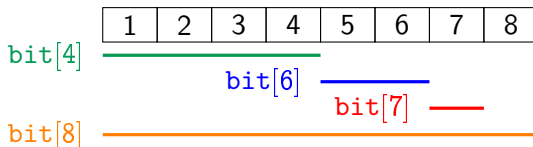
- $\text{bit}[i]$ stores the interval

$$[i - \text{lowbit}(i) + 1, i].$$

Example: 4-bit two's complement

$$\begin{array}{rcl} 6 & = & 0110_2 \\ -6 & = & 1010_2 \\ \hline \& & = 0010_2 = 2 \end{array}$$

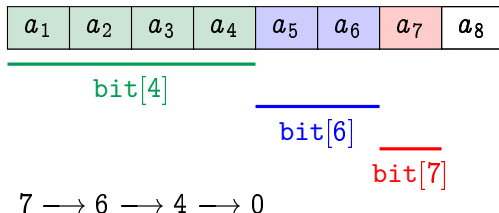
The lowest 1-bit stays aligned; every higher bit is flipped.



A prefix is partitioned into Fenwick ranges

- To compute $a_1 + \dots + a_7$, use three disjoint stored sums:

$$\text{bit}[7] + \text{bit}[6] + \text{bit}[4].$$



- Each step removes the lowest set bit: $i \leftarrow i - \text{lowbit}(i)$.



Use one-based, zero-initialized storage

```
1  int n;  
2  vector<long long> bit, a;  
3  
4  void init(int size) {  
5      n = size;  
6      bit.assign(n + 1, 0);  
7      a.assign(n + 1, 0);  
8  }
```

- Valid positions are $1, \dots, n$; $\text{bit}[0]$ is unused.
- $\text{prefix_sum}(i)$ accepts $0 \leq i \leq n$, with $\text{prefix_sum}(0) = 0$.
- $\text{add}(i, \text{delta})$ requires $1 \leq i \leq n$; use it to load each initial value.



A prefix query walks downward

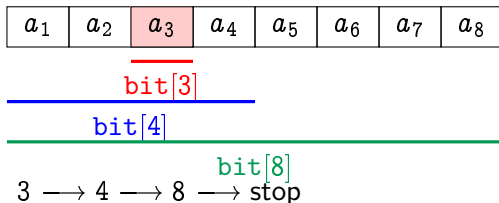
```
1  int lowbit(int i) { return i & -i; }
2
3  long long prefix_sum(int i) {
4      long long result = 0;
5      while (i > 0) {
6          result += bit[i];
7          i -= lowbit(i);
8      }
9      return result;
10 }
```

- Each step clears one set bit of i .
- An index has at most $\lfloor \log_2 N \rfloor + 1$ bits, so the query takes $O(\log N)$ time.



A point update visits every range containing it

- Adding Δ to a_3 changes $\text{bit}[3]$, $\text{bit}[4]$, and $\text{bit}[8]$.



- Each step moves to the next larger stored range containing p :
 $i \leftarrow i + \text{lowbit}(i)$.



A point update walks upward

```
1 void add(int i, long long delta) {  
2     a[i] += delta;  
3     while (i <= n) {  
4         bit[i] += delta;  
5         i += lowbit(i);  
6     }  
7 }
```

- Every visited entry represents a range containing the updated position.
- The index gains a higher set bit at each step, so the update takes $O(\log N)$ time.
- Build the tree by calling `add(i, value)` for every input value in $O(N \log N)$ time.



Two prefix sums recover any range sum

- The prefix through r contains both the requested range and the prefix before it:

$$\text{range_sum}(l, r) = \text{prefix_sum}(r) - \text{prefix_sum}(l - 1).$$

- Both prefix queries take $O(\log N)$ time, so the whole range query does too.
- This subtraction works because sums have an inverse.

Important limitation

A basic Fenwick tree cannot recover an arbitrary range maximum from two prefix maxima.



Assignment updates become additive deltas

- If an operation says $a_p \leftarrow x$, store the current array value and add only the difference.

```
1 void assign(int p, long long x) {  
2     add(p, x - a[p]);  
3 }
```

- Initial values, later additions, and assignments all update the same Fenwick invariant.



Choose the simplest structure that supports the operation

	Fenwick tree	Segment tree
Best fit	Prefix/range sums with point updates	Flexible associative range queries
Query	Prefix first; range by inverse	Any interval directly
Space	$O(N)$: $N + 1$ tree slots; assignments also keep <code>a[]</code>	$O(N)$: $2N - 1$ nodes here
Implementation	Short iterative loops	More structure and code

- Use a Fenwick tree when its prefix-based operation is enough.
- Use a segment tree when the combine rule or updates need more flexibility.



The lowbit controls both directions

Operation	Index movement	Cost
Prefix sum	$i \leftarrow i - \text{lowbit}(i)$	$O(\log N)$
Point add	$i \leftarrow i + \text{lowbit}(i)$	$O(\log N)$
Range sum	Two prefix sums	$O(\log N)$
Storage	One partial-sum array	$O(N)$

Core invariant

$\text{bit}[i]$ stores the range ending at i whose length is $\text{lowbit}(i)$.



1 Segment Tree Basics

2 Fenwick Trees

3 Lazy Propagation

4 Sparse Tables



Lazy Propagation

Range updates expose the missing case

Problem (Dynamic Range Maximum)

Maintain an array while processing two operations:

- add Δ to every value in a range $[q_l, q_r]$;
 - report the maximum value in a range $[l, r]$.
-
- Repeating a point update for every affected position can take $O(N \log N)$ time.
 - A fully covered node already represents the whole update range.
 - **Lazy propagation** updates that node now and postpones work below it.



A lazy tag records work owed to descendants

```
1  struct Node {  
2      Node *lc, *rc;  
3      int mx, lazy = 0;  
4      void pull() { mx = max(lc->mx, rc->mx); }  
5  };
```

- `mx` is always correct for the node's entire interval.
- `lazy` is an addition already included in `mx`, but not yet sent to the children.
- During `build()`, every node starts with `lazy = 0`.



Apply a full-cover update without descending

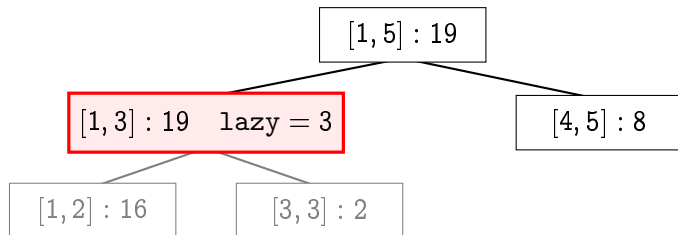
```
1 void apply(Node *nd, int delta) {  
2     nd->mx += delta;  
3     nd->lazy += delta;  
4 }
```

- Adding Δ to every value also adds Δ to the interval maximum.
- The stored maximum changes immediately, so a full-cover query remains correct.
- Additive updates compose by adding their tags.



One tag can represent an entire updated subtree

- Start with $a = [1, 16, 2, 8, 4]$ and add 3 to the whole range $[1, 3]$.



- The red node and its ancestors are correct; its gray descendants still owe $+3$.

Push before descending to children

```
1 void push(Node *nd) {  
2     if (nd->lazy == 0 || nd->lc == nullptr) return;  
3     apply(nd->lc, nd->lazy);  
4     apply(nd->rc, nd->lazy);  
5     nd->lazy = 0;  
6 }
```

- `push()` transfers the pending update to both children exactly once.
- The leaf guard avoids accessing missing children.
- After the transfer, the parent has no remaining work to defer.



Point assignment must respect pending tags

```
1 void modify(Node *nd, int val, int p, int l, int r) {
2     if (l == r) {
3         nd->mx = val;
4         nd->lazy = 0;
5         return;
6     }
7     push(nd);
8     int m = (l + r) / 2;
9     if (p <= m) modify(nd->lc, val, p, l, m);
10    else modify(nd->rc, val, p, m + 1, r);
11    nd->pull();
12 }
```

- Push before descending; assignment replaces every older update at the leaf.



A range update has three overlap cases

- Update $[q_l, q_r]$ while the current node represents $[L, R]$:
 - 1 **Disjoint:** return without changing anything.
 - 2 **Fully covered:** call `apply()` and stop descending.
 - 3 **Partial overlap:** call `push()`, update both children, then `pull()`.
- Full-cover nodes are the reason one range update does not visit every leaf.



Implement a lazy range-add update

```
1 void range_add(Node *nd, int ql, int qr, int delta,
2               int l, int r) {
3     if (r < ql || qr < l) return;
4     if (ql <= l && r <= qr) {
5         apply(nd, delta);
6         return;
7     }
8     push(nd);
9     int m = (l + r) / 2;
10    range_add(nd->lc, ql, qr, delta, l, m);
11    range_add(nd->rc, ql, qr, delta, m + 1, r);
12    nd->pull();
13 }
```



Queries push before entering a partial overlap

```
1  int range_max(Node *nd, int ql, int qr,
2      int l, int r) {
3      if (r < ql || qr < l) return neg_inf;
4      if (ql <= l && r <= qr) return nd->mx;
5      push(nd);
6      int m = (l + r) / 2;
7      return max(range_max(nd->lc, ql, qr, l, m),
8                  range_max(nd->rc, ql, qr, m + 1, r));
9  }
```

- A full-cover query can use `mx` without pushing.
- A partial query must push first so both child summaries are current.



Both operations stay logarithmic

Operation	Cost	Why
Build	$O(N)$	One visit per node
Range add	$O(\log N)$	Stop at full-cover nodes
Range maximum	$O(\log N)$	Visit a logarithmic decomposition
Storage	$O(N)$	One extra tag per node

- Lazy propagation changes when work is performed, not the segment tree's asymptotic space.



The tag must match the update operation

- Range addition composes by adding tags: $+x$ followed by $+y$ becomes $+(x + y)$.
- Range assignment is different: a newer assignment replaces an older pending assignment.
- Mixed update types may need multiple tags and a defined composition order.

Common implementation mistakes

Forgetting to push before descending, forgetting to pull after a partial update, or changing the tag without changing the stored summary.



Lazy propagation follows one flow

Overlap	Action
Disjoint	Ignore the node
Full	Apply to the summary and compose the tag
Partial	Push $\rightarrow$ recurse $\rightarrow$ pull

Core invariant

Every node summary includes its own pending tag, even when its descendants do not yet include that update.



1 Segment Tree Basics

2 Fenwick Trees

3 Lazy Propagation

4 Sparse Tables



Sparse Tables

Static queries reward preprocessing

Problem (Static Range Minimum)

Given a fixed array $a_0, \dots, a_{N-1}$, answer queries asking for

$$\min(a_l, a_{l+1}, \dots, a_r).$$

The array never changes after preprocessing.

- Scanning every query can take $O(NQ)$ time in the worst case.
- A segment tree answers each query in $O(\log N)$ time.
- A **sparse table** spends $O(N \log N)$ time up front, then answers each range minimum in $O(1)$ time.



Store every power-of-two interval

- For every length $L \geq 1$, let

$$k = \lfloor \log_2 L \rfloor, \quad 2^k \leq L < 2^{k+1}.$$

- Define $\text{st}[k][i]$ as the minimum of the length- 2^k interval starting at i .

Level	Length	Intervals stored
$k = 0$	1	Every single value
$k = 1$	2	Every adjacent pair
$k = 2$	4	Every consecutive block of four



Each larger block reuses two equal halves

- A length- 2^k interval splits into two adjacent intervals of length 2^{k-1} :

$$\text{st}[k][i] = \min(\text{st}[k-1][i], \text{st}[k-1][i + 2^{k-1}]).$$

	Values	Left half	Right half
$a[0 \dots 3]$	7, 2, 3, 0	$\min(7, 2) = 2$	$\min(3, 0) = 0$
$\text{st}[2][0]$	0	$\min(2, 0) = 0$	

- Every table entry takes $O(1)$ time once the previous level exists.



Precompute floor logs for every length

```
1  vector<int> lg(n + 1, 0);
2  for (int len = 2; len <= n; ++len) {
3      lg[len] = lg[len / 2] + 1;
4  }
```

- Integer division by two removes the lowest binary digit.
- Therefore $\text{lg}[\text{len}]$ equals $\lfloor \log_2 \text{len} \rfloor$.
- The same array chooses the number of sparse-table levels and answers every query's logarithm in $O(1)$ time.



Build from short intervals to long ones

```
1  int levels = lg[n] + 1;
2  vector<vector<int>> st(levels, vector<int>(n));
3  st[0] = a;
4
5  for (int k = 1; k < levels; ++k) {
6      int len = 1 << k;
7      for (int i = 0; i + len <= n; ++i) {
8          st[k][i] = min(st[k - 1][i],
9                          st[k - 1][i + len / 2]);
10     }
11 }
```

- Level k contains $N - 2^k + 1$ valid starting positions.



Two equal blocks cover any query interval

- Query $[2, 7]$ has length 6, so choose $k = \lfloor \log_2 6 \rfloor = 2$ and block length $2^k = 4$.

Block	Interval	Table entry
Leftmost length 4	$[2, 5]$	st [2] [2]
Rightmost length 4	$[4, 7]$	st [2] [4]

- Their union covers $[2, 7]$; the middle positions 4 and 5 appear in both blocks.
- Repeating values does not change a minimum, so the overlap is harmless.



Range minimum uses two lookups

```
1  int range_min(int l, int r) {  
2      int len = r - l + 1;  
3      int k = lg[len];  
4      int right_start = r - (1 << k) + 1;  
5      return min(st[k][l], st[k][right_start]);  
6  }
```

- The left block begins at l .
- The right block ends at r , so it begins at $r - 2^k + 1$.
- No loop depends on the query length; the query takes $O(1)$ time.



Overlap is safe only for idempotent operations

The two-block query repeats the overlap, so it needs $f(x, x) = x$.

Operation	Idempotent	Overlap effect
Minimum / maximum	Yes	Duplicates do not matter
GCD, bitwise AND / OR	Yes	Duplicates do not matter
Sum	No	Overlap is counted twice
XOR	No	Duplicate values cancel

- Associativity builds the levels; idempotence makes the overlapping $O(1)$ query correct.



Preprocessing buys constant-time queries

Operation	Cost	Why
Build	$O(N \log N)$	N entries across $O(\log N)$ levels
Range minimum	$O(1)$	Two table entries
Storage	$O(N \log N)$	One row per power of two
Point update	Not supported	Many precomputed blocks change

- Sparse tables are strongest when the array is fixed and there are many queries.



Match the structure to the update model

	Sparse table	Fenwick tree	Segment tree
Best fit	Static idempotent queries	Dynamic prefix-based sums	Flexible dynamic queries
Query	$O(1)$ RMQ	$O(\log N)$	$O(\log N)$
Update	None	$O(\log N)$ point	$O(\log N)$ point or range
Space	$O(N \log N)$	$O(N)$	$O(N)$

- If updates exist, the sparse table's faster query no longer compensates for rebuilding it.



Boundary mistakes cause most bugs

- This implementation uses zero-based indices and inclusive queries $[l, r]$.
- Compute the length as $r - l + 1$, not $r - l$.
- Start the right block at $r - 2^k + 1$, not $r - 2^k$.
- Allocate $\lfloor \log_2 N \rfloor + 1$ levels and build only intervals that stay inside the array.
- Use the overlapping $O(1)$ query only for an idempotent operation.

Quick boundary tests

Check a one-element query, the whole array, and intervals whose lengths are exactly powers of two.



Static RMQ becomes constant-time lookup

- 1 Store the answer for every power-of-two interval.
- 2 Build each level from two adjacent halves of the previous level.
- 3 Cover $[l, r]$ with its leftmost and rightmost blocks of length $2^{\lfloor \log_2(r-l+1) \rfloor}$.
- 4 Combine those two answers with an idempotent operation.

Core decision rule

Use a sparse table when the array is immutable and constant-time range queries justify $O(N \log N)$ preprocessing and storage.



Practice roadmap

- [Dynamic Range Sum Queries \(CSES 1648\)](#) – point updates with a Fenwick tree or segment tree.
- [Dynamic Range Minimum Queries \(CSES 1649\)](#) – the segment-tree pattern with a different combine rule.
- [Range Update Queries \(CSES 1651\)](#) – range additions with point queries; compare a lazy tree with a simpler difference-based approach.
- [Static Range Minimum Queries \(CSES 1647\)](#) – sparse-table preprocessing and $O(1)$ queries.

Suggested order

Solve the point-update problems first, then try range updates and compare each implementation with the structure's invariant.



Thank you

Thanks for listening.

© 2026 Sean. Please do not repost or redistribute without permission.

